CCP Reference and Known-Truth Validation Tutorial

Hotz-Miller Inversion and Nested Pseudo-Likelihood

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/ccp.html **Source:** https://github.com/rawatpranjal/EconIRL/tree/main

1 Purpose

CCP, the conditional choice probability estimator, is the structural dynamic discrete choice estimator that replaces repeated Bellman fixed-point solves with first-stage policy inversion. Instead of solving the dynamic programming problem for every candidate payoff vector, CCP first estimates $\hat{\pi}(a \mid s)$ from observed choices and uses the Hotz-Miller inversion to recover continuation values. Aguirregabiria and Mira's NPL algorithm then iterates this mapping until it approximates the MLE fixed point.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and Type A/B/C counterfactual oracle objects. The validation question is whether the CCP/NPL implementation recovers these known objects on a medium-scale finite panel.

2 Source Papers

The primary source is Hotz and Miller (1993), which showed that conditional choice probabilities can invert the dynamic choice problem under additive separability, conditional independence, and type-I extreme-value shocks. Aguirregabiria and Mira (2002) introduced nested pseudo-likelihood, or NPL: given a policy, estimate payoffs; given payoffs, update the policy; repeat. Their sequence nests the one-step Hotz-Miller estimator at $K = 1$ and the NFXP/MLE fixed point in the limit. Magnac and Thesmar (2002) are the identification warning: the discount factor, scale, and reward location need normalization or exclusion restrictions.

3 When to Use CCP

Use CCP when the state and action spaces are tabular, transitions are known or first-stage estimated, empirical action support is strong, and a full NFXP inner solve at every outer step is too expensive. CCP is especially useful as a structural workhorse when a nonparametric first-stage policy can be estimated well.

Do not use CCP as a recovery benchmark when many states are rarely observed or one action is almost never chosen. The first-stage empirical CCPs are the estimator's foundation; if they are poorly supported, the Hotz-Miller inversion amplifies the noise.

4 Model

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

Flow payoff is linear in known features:

$$u_\theta(s,a) = \phi(s,a)^\top \theta.$$

Transitions are

$$P_a(s,s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a),$$

the discount factor is β, and the type-I extreme-value shock scale is σ.

Given a policy π, define the policy-weighted transition matrix

$$F_\pi(s,s') = \sum_a \pi(a \mid s) P_a(s,s')$$

and the logit emax correction

$$e_\pi(s,a) = \gamma_{\text{Euler}} - \log \pi(a \mid s).$$

Hotz-Miller inversion writes the integrated value under policy π as

$$\bar{V}_\pi = (I - \beta F_\pi)^{-1} \sum_a \pi(a \mid s)\{u_\theta(s,a) + e_\pi(s,a)\}.$$

For linear rewards, this separates into augmented features:

$$\bar{V}_\pi(s) = W_\phi(s)^\top \theta + W_e(s),$$

where

$$W_\phi = (I - \beta F_\pi)^{-1} \sum_a \pi(a \mid s)\phi(s,a), \qquad W_e = (I - \beta F_\pi)^{-1} \sum_a \pi(a \mid s) e_\pi(s,a).$$

The choice-specific value used in the pseudo-likelihood is

$$\tilde{Q}_\theta(s,a;\pi) = \phi(s,a)^\top \theta + \beta \sum_{s'} P_a(s,s')\{W_\phi(s')^\top \theta + W_e(s')\}.$$

The K-stage CCP/NPL estimator is

$$\hat{\theta}_K = \arg\max_\theta \sum_{i,t} \log \frac{\exp(\tilde{Q}_\theta(s_{it}, a_{it}; \hat{\pi}^{K-1})/\sigma)}{\sum_b \exp(\tilde{Q}_\theta(s_{it}, b; \hat{\pi}^{K-1})/\sigma)}.$$

5 Estimator Mechanics

The implementation follows five steps.

1. Estimate empirical CCPs with Laplace smoothing:

$$\hat{\pi}_0(a \mid s) = \frac{N(s,a) + \delta}{N(s) + |\mathcal{A}|\delta}.$$

2. Compute $F_{\hat{\pi}}$, $e_{\hat{\pi}}$, W_ϕ, and W_e once per NPL step.
3. Build augmented features and maximize a standard multinomial logit pseudo-likelihood in θ.
4. Update CCPs from the fitted choice-specific values.
5. Repeat for K NPL iterations, then compute standard errors from the full Bellman-constrained likelihood Hessian.

The important implementation detail is the cost shift. NFXP solves a Bellman fixed point inside every likelihood evaluation. CCP pays one matrix inversion per NPL step, then each optimizer step is just a logit gradient on augmented features. In EconIRL, the augmented-feature logit kernel is JIT-compiled once and reused across NPL steps.

6 Identification Checks

CCP inherits the same payoff identification restrictions as NFXP and adds first-stage support requirements.

1. **Reward rank.** The action-dependent feature matrix must have full column rank after dropping zero rows.
2. **Action support.** Every action must have enough empirical mass. Near-zero CCPs make $e_\pi(s,a) = \gamma - \log \pi(a \mid s)$ unstable.
3. **State coverage.** The policy is nonparametrically estimated by state. Sparse states should be pooled, smoothed, or excluded from a main validation claim.
4. **Normalization.** The reward scale is fixed at $\sigma = 1$, and the synthetic DGP uses an exit action and absorbing state as a zero-reward anchor.
5. **Transition separation.** This tutorial supplies the true transition tensor, so the validation isolates the CCP payoff estimator rather than transition-estimation error.

7 Known-Truth DGP

The canonical CCP validation cell is `canonical_low_action`. It is the same low-dimensional action-dependent structural benchmark used for NFXP.

Table 1: Canonical CCP DGP.

Quantity	Value
Regular states	20
Absorbing states	1
Total states	21
Actions	3
Exit action	2
Discount factor	0.95
Shock scale	1.0
Reward mode	action-dependent
Reward dimension	low
State mode	low-dimensional
Simulated individuals	2,000
Periods per individual	80
Observations	160,000

The action-dependent features are

$$\phi(s,a) = \begin{cases} (1, x_s, 0, 0) & a = 0, \\ (0, 0, 1, x_s) & a = 1, \\ (0, 0, 0, 0) & a = 2. \end{cases}$$

The exit action has zero reward and sends the process to the absorbing state. The true parameter vector is:

Table 2: True reward parameters.

Parameter	Truth
`action_0_intercept`	0.10
`action_0_progress`	0.50
`action_1_intercept`	0.00
`action_1_progress`	-0.20

8 Pre-Estimation Diagnostics

Run diagnostics before fitting. For CCP, support diagnostics are not optional: the estimator starts from empirical CCPs.

```
from experiments.known_truth import (
    SimulationConfig,
    build_known_truth_dgp,
    get_cell,
    run_pre_estimation_diagnostics,
    simulate_known_truth_panel,
)

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(
    dgp,
    SimulationConfig(n_individuals=2000, n_periods=80, seed=42),
)
diagnostics = run_pre_estimation_diagnostics(dgp, panel)

assert diagnostics.passed
assert diagnostics.feature_rank == diagnostics.num_features
assert diagnostics.is_action_dependent
assert diagnostics.state_action_coverage == 1.0
```

9 Run the Tutorial

From the repository root:

```
PYTHONPATH=src:. python -m experiments.known_truth \
    --estimator CCP \
    --cell-id canonical_low_action \
    --output-dir outputs/known_truth \
    --show-progress \
    --verbose
```

The command builds the DGP, simulates the panel with `tqdm` progress, runs pre-estimation diagnostics, fits K=10 CCP/NPL with finite standard errors, evaluates reward/value/Q/policy recovery, solves Type A/B/C counterfactual oracles, writes one `result.json`, and raises if any non-smoke hard gate fails.

Direct estimator usage:

```
from experiments.known_truth import (
    build_known_truth_dgp,
    get_cell,
    simulate_known_truth_panel,
)
from econirl.estimation.ccp import CCPEstimator

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(dgp, cell.simulation_config)

estimator = CCPEstimator(
    num_policy_iterations=10,
    ccp_min_count=1,
    ccp_smoothing=1e-6,
    outer_tol=1e-6,
```

```
    outer_max_iter=500,
    se_method="robust",
    compute_hessian=True,
    verbose=True,
)

summary = estimator.estimate(
    panel=panel,
    utility=dgp.utility(),
    problem=dgp.problem,
    transitions=dgp.transitions,
)
```

10 Generated Validation Results

This section is generated by `ccp_run.py` from the `experiments.known_truth` harness and the canonical known-truth DGP.

10.1 Pre-Estimation Checks

Table 3: Pre-estimation checks from the known-truth CCP run.

Check	Value	Status
Feature rank	4 / 4	pass
Feature condition number	4.512	pass
Transition row error	2.42×10^{-8}	pass
Observed states	21 / 21	pass
State-action coverage	1.000	pass
Action shares	0.345, 0.330, 0.325	pass
Minimum action share	0.325	pass
Minimum positive CCP	0.153	pass
Exit/absorbing anchor	true	pass

10.2 Estimator Fit

The medium-scale validation run uses 2,000 individuals, 80 periods per individual, and 160,000 observations.

Table 4: Known-truth CCP/NPL run summary.

Quantity	Value
NPL iterations completed	10
NPL delta criterion met	false
Log-likelihood	-174875.9531
Estimation time	30.04 seconds
Standard errors finite	true

Table 5: Parameter recovery.

Parameter	Truth	Estimate	SE	Error
`action_0_intercept`	0.100000	0.083895	0.035995	-0.016105
`action_0_progress`	0.500000	0.528519	0.039855	0.028519
`action_1_intercept`	0.000000	-0.014460	0.117503	-0.014460
`action_1_progress`	-0.200000	-0.200513	0.158931	-0.000513

Table 6: Recovery metrics.

Metric	Value
Parameter RMSE	0.017903
Parameter relative RMSE	0.065372
Parameter cosine similarity	0.998867
Reward RMSE	0.009694
Value RMSE	0.019438
Q RMSE	0.022432
Policy KL	9.21×10^{-5}
Policy total variation	0.005697
Policy max state L1	0.018905

Table 7: K-stage CCP/NPL comparison on the same panel.

Estimator	Iter.	Rel. RMSE	Cosine	Policy TV	Value RMSE	Time (s)
Hotz-Miller K=1	1	0.058259	0.999211	0.005539	0.016469	2.12
NPL K=3	3	0.065298	0.998871	0.005694	0.019416	6.88
NPL K=10	10	0.065372	0.998867	0.005697	0.019438	30.04

In this canonical finite sample, the one-step Hotz-Miller row already recovers the main policy well because the empirical CCPs have full support. NPL iterations are still the estimator used for the hard validation because they move the pseudo-likelihood mapping toward the MLE fixed point and provide the appropriate structural object for standard errors.

Table 8: Hard recovery gates.

Gate	Threshold	Value	Status
npl_iterations	≥ 5.000000	10.000000	pass
standard_errors_finite	true	true	pass
parameter_cosine	≥ 0.980000	0.998867	pass
parameter_relative_rmse	≤ 0.150000	0.065372	pass
policy_tv	≤ 0.030000	0.005697	pass
value_rmse	≤ 0.100000	0.019438	pass
q_rmse	≤ 0.100000	0.022432	pass
type_a_regret	≤ 0.050000	0.000213	pass
type_b_regret	≤ 0.050000	0.000362	pass
type_c_regret	≤ 0.050000	0.000086	pass

10.3 Counterfactual Recovery

Table 9: Counterfactual recovery against known oracle objects.

Counterfactual	Policy TV	Policy KL	Value RMSE	Regret
Type A	0.005109	7.56×10^{-5}	0.000238	0.000213
Type B	0.005457	8.20×10^{-5}	0.000363	0.000362
Type C	0.003548	3.56×10^{-5}	0.000114	0.000086

11 Debugging Gaps

If CCP fails to recover truth, debug in this order.

1. **First-stage support.** Check observed states, state-action coverage, minimum action share, and minimum positive CCP.
2. **One-action states.** If a state has only one observed action, Hotz-Miller inversion is extrapolating that state's counterfactual action values.
3. **Smoothing.** Too little smoothing makes logs unstable; too much smoothing biases the first-stage policy toward uniform choice.
4. **Feature rank.** If rank is below the number of reward parameters, θ is not identified.
5. **Normalization.** If the exit/absorbing anchor is invalid, reward levels can drift.
6. **NPL iterations.** A K=1 one-step estimator can be consistent, but the validation claim uses K=10 and hard gates.
7. **Value convention.** CCP inversion includes the Euler-constant emax correction internally. EconIRL reports values in the package's soft Bellman convention by evaluating the recovered policy under the recovered reward, so known-truth value comparisons are meaningful.
8. **Soft gates.** Do not accept a non-smoke run that only warns. Known-truth validation should pass hard gates or raise.

12 Practical Defaults

Recommended non-smoke validation settings:

```
CCPEstimator(
    num_policy_iterations=10,
    ccp_min_count=1,
    ccp_smoothing=1e-6,
    convergence_tol=1e-6,
    outer_tol=1e-6,
    outer_max_iter=500,
    se_method="robust",
    compute_hessian=True,
)
```

Use smoke settings only for code-path tests. A smoke test shows that the estimator runs and returns finite objects; it is not evidence of recovery.

13 Implementation Paths

- Estimator: `src/econirl/estimation/ccp.py`
- Known-truth harness: `experiments/known_truth.py`
- Fast validation tests: `tests/test_known_truth.py`
- Master plan: `papers/econirl_package_jss/plans/README.md`
- CCP alignment audit: `papers/econirl_package_jss/plans/alignment/03_ccp.md`

14 References

Hotz, V. Joseph, and Robert A. Miller (1993). Conditional Choice Probabilities and the Estimation of Dynamic Models. *The Review of Economic Studies*, 60(3), 497–529.

Aguirregabiria, Victor, and Pedro Mira (2002). Swapping the Nested Fixed Point Algorithm: A Class of Estimators for Discrete Markov Decision Models. *Econometrica*, 70(4), 1519–1543.

Magnac, Thierry, and David Thesmar (2002). Identifying Dynamic Discrete Decision Processes. *Econometrica*, 70(2), 801–816.